

Relational Algebra



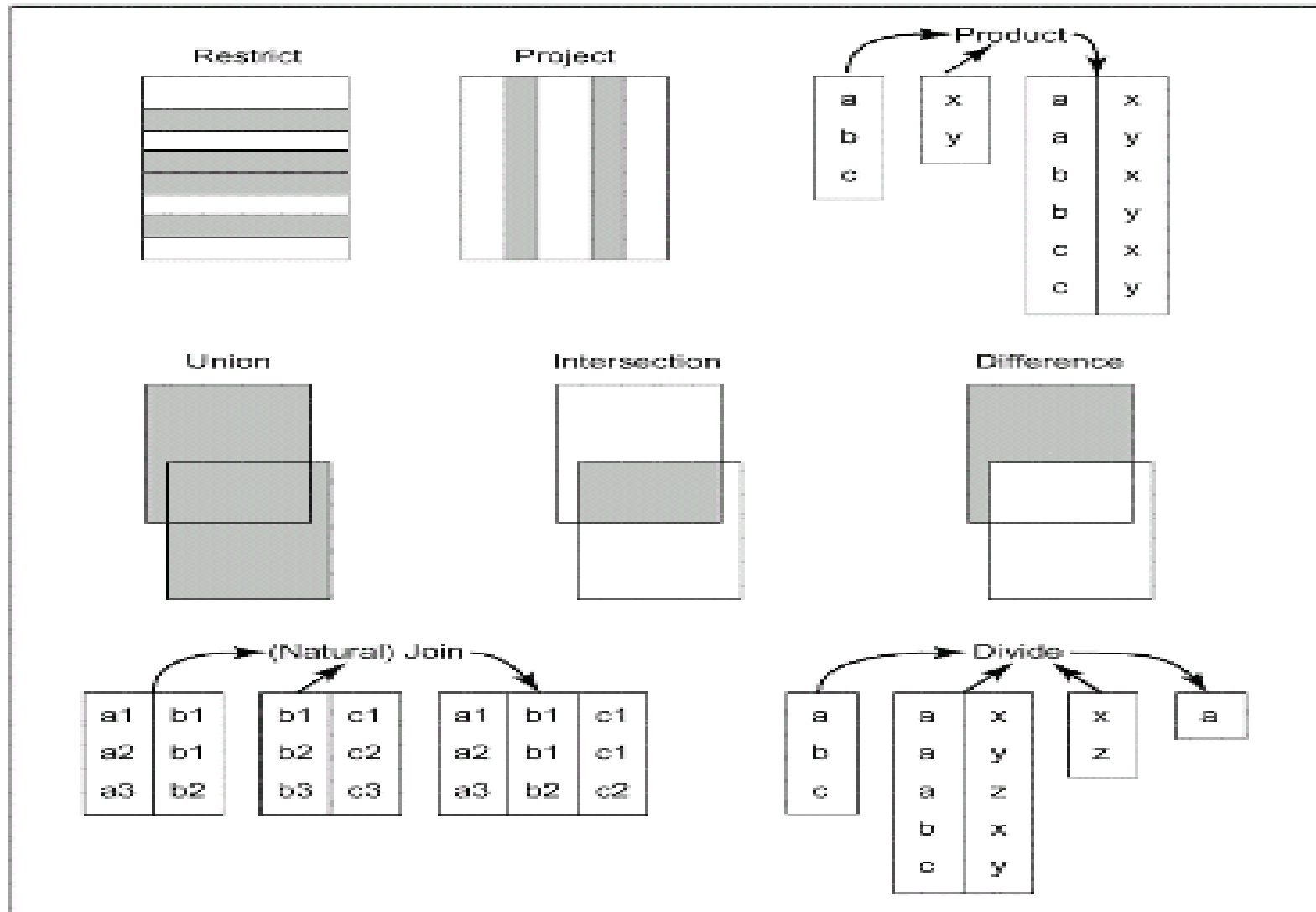
What will you learn?

- Codd's original eight operators
- closure property
- semantics
- what is the Algebra for?

1. Introduction

- **Relational algebra** is basically just a set of operators that take relations as their operands and return a relation as their result.
- Codd's original eight operators
 - **The traditional set operators:**
 - Union (并)
 - Intersection (交)
 - Difference (差)
 - Cartesian product (迪卡尔积)
 - **The special relational operators:**
 - restrict/select (选择)
 - Project (投影)
 - Join (连接)
 - Divide (除)

Fig. 6.1 The original eight operators (overview)

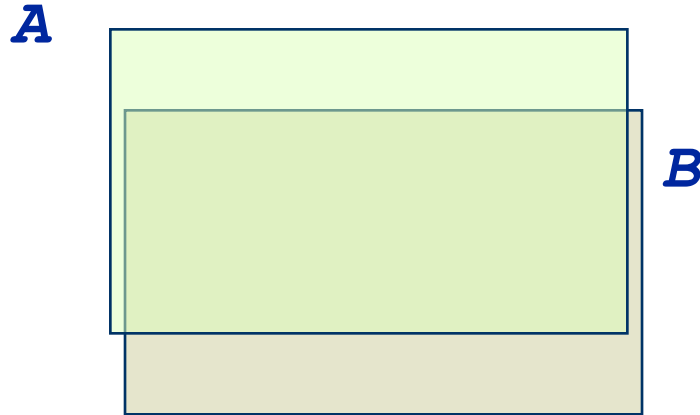


2. Closure property

- Closure property:
 - The output from any given relational operation is another relation.
 - Nested relational expressions
- RENAME operator:
 - Rename attributes within a specified relation.
e.g. **S RENAME CITY AS SCITY**

S#	SNAME	STATUS	SCITY
S1	Smith	20	London
S2	Jones	10	Paris
S3	Blake	30	Paris
S4	Clark	20	London
S5	Adams	30	Athens

3. Semantics (关系操作的语义) -- Union



Two relations are **type-compatible** if they have the same set of attribute names which are defined on the same domains

The union of two (type-compatible) relations A and B :

$$A \text{ UNION } B = \{t \mid t \in A \vee t \in B\}$$

- is a relation with the same heading as each of A and B and with a
- body consisting of the set of all tuples t belonging to A or B or both.

Example of Union

Airport1

airport_name	country_name
London/Gk	England
Naples	Italy
Pisa	Italy

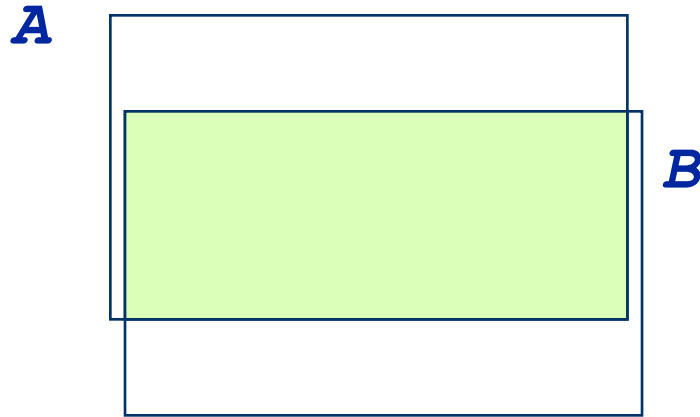
Airport2

airport_name	country_name
London/Gk	England
London/Hw	England
Manchester	England

Airport1 UNION Airport2

airport_name	country_name
London/Gk	England
Naples	Italy
Pisa	Italy
London/Hw	England
Manchester	England

3. Semantics-- Intersection



The intersection of two (type-compatible) relations A and B :

$$A \text{ INTERSECTION } B = \{t \mid t \in A \wedge t \in B\}$$

- is a relation with the same heading as each of A and B and with a

body consisting of the set of all tuples t belonging to both A and B .

Example of Intersection

Airport1

airport_name	country_name
London/Gk	England
Naples	Italy
Pisa	Italy

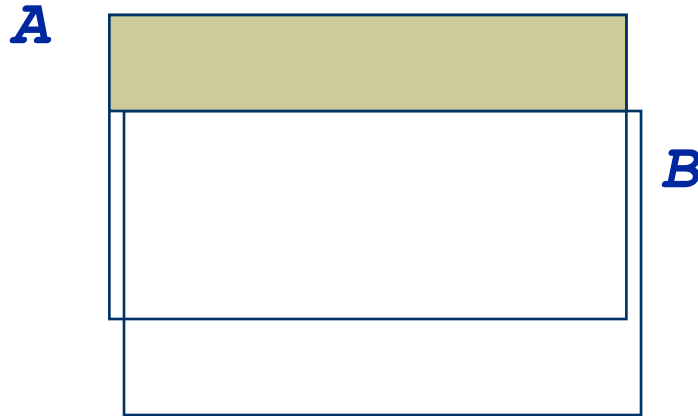
Airport2

airport_name	country_name
London/Gk	England
London/Hw	England
Manchester	England

Airport1 INTERSECTION Airport2

airport_name	country_name
London/Gk	England

3. Semantics-- Difference



The difference of two (type-compatible) relations A and B in that order:

$$A \text{ MINUS } B = \{t \mid t \in A \wedge !t \in B \}$$

- is a relation with the same heading as each of A and B and with a

body consisting of the set of all tuples t belonging to A and not to B .

Example of Difference

Airport1

airport_name	country_name
London/Gk	England
Naples	Italy
Pisa	Italy

Airport2

airport_name	country_name
London/Gk	England
London/Hw	England
Manchester	England

Airport1 MINUS Airport2

airport_name	country_name
Naples	Italy
Pisa	Italy

What about Airport2 MINUS Airport1?

Example of Union/Intersection/Difference

A				B			
S#	SNAME	STATUS	CITY	S#	SNAME	STATUS	CITY
S1	Smith	20	London	S1	Smith	20	London
S4	Clark	20	London	S2	Jones	10	Paris

a. Union (A UNION B)				S#	SNAME	STATUS	CITY
				S1	Smith	20	London
				S4	Clark	20	London
				S2	Jones	10	Paris

b. Intersection (A INTERSECT B)				S#	SNAME	STATUS	CITY
				S1	Smith	20	London

c. Difference (A MINUS B)				S#	SNAME	STATUS	CITY
				S4	Clark	20	London

d. Difference (B MINUS A)				S#	SNAME	STATUS	CITY
				S2	Jones	10	Paris

Fig. 6.2 Union, intersection, and difference examples

3. Semantics - Product

a1	TIMES	b1	=	a1	b1
a2		b2		a1	b2
				a2	b1
				a2	b2

Let relations A and B have headings $\{X\}$ and $\{Y\}$ respectively.

The Cartesian product of A and B where they have no common attribute names:

$$A \text{ TIMES } B = \bigwedge \{ab \mid a \in A \wedge b \in B\}$$

- is a relation with the heading that is the sum of the headings of A and B and
body consisting of the set of all tuples t such that t is the coalescing of a tuple a in A and a tuple b in B .

More about Product...

- The **cardinality** of the result of A TIMES B is the product of the cardinalities of A and B
- The **degree** of the result is the sum of the degrees of A and B
- The Cartesian product is not very important in practice
- The result is often large and space-costly but produces no more information than there was in the input

Example of Product...

A		B	
S#		P#	
S1		P1	
S2		P2	
S3		P3	
S4		P4	
S5		P5	
		P6	

Cartesian product (A TIMES B)											
S#	P#										
S1	P1	S2	P1	S3	P1	S4	P1	S5	P1		
S1	P2	S2	P2	S3	P2	S4	P2	S5	P2		
S1	P3	S2	P3	S3	P3	S4	P3	S5	P3		
S1	P4	S2	P4	S3	P4	S4	P4	S5	P4		
S1	P5	S2	P5	S3	P5	S4	P5	S5	P5		
S1	P6	S2	P6	S3	P6	S4	P6	S5	P6		
..	..	..	..	..	..	..	..				

Fig. 6.3 Cartesian product example

3. Semantics - Restrict

A

The restriction of relation *A* on the condition:

A WHERE condition

-is a relation with the same heading as *A* and with a
-body consisting
of the set of all tuples *t* of *A* such that the condition is true
for *t*

Restriction Condition

$A \text{ WHERE } X \theta Y$

X, Y are attributes of A , and must be defined on the same domain

θ is any simple scalar comparison operator like $=, \neq, >, \geq$ etc.

In fact, the restriction condition can be an arbitrary Boolean combination of simple comparisons connected by AND, OR, NOT.

Take note: Unlike the traditional set operators, restrict takes a relation and an expression as input, and produces a relation as output. Some concepts like commutativity and associativity no longer applicable.

Example of Restrictions

Airports

airport_name	country_name
Naples	Italy
London/Gk	England
London/Hw	England
Pisa	Italy
Venice/MP	Italy
Manchester	England
Kai Tak	Hong Kong
Changi	Singapore

Airports WHERE airport_name = 'London/GK'

airport_name	country_name
London/Gk	England

Airports WHERE country_name = 'England'

airport_name	country_name
London/Gk	England
London/Hw	England
Manchester	England

Example of Restrictions

S WHERE CITY = 'London'

S#	SNAME	STATUS	CITY
S1	Smith	20	London
S4	Clark	20	London

P WHERE WEIGHT <
WEIGHT (14.0)

P#	PNAME	COLOR	WEIGHT	CITY
P1	Nut	Red	12.0	London
P5	Cam	Blue	12.0	Paris

SP WHERE S# = S# ('S6')
OR P# = P# ('P7')

S#	P#	QTY

Fig. 6.4 Restriction examples

3. Semantics - Project

A

The projection of relation A on attributes $X, Y, \dots, Z$:

$A \{X, Y, \dots, Z\}$

- is a relation with the heading $\{X, Y, \dots, Z\}$ and with the a body

consisting of the set of all tuples $\{X:x, Y:y, \dots, Z:z\}$ such that a

tuple appears in A with X -value x , Y -value y , $\dots$, Z -value z .

More about Projection

The projection operator produces the output table by 1) eliminating all attributes not specified, and 2) eliminating any duplicated tuples

A -- identity projection

A{ } -- nullary projection (both not very useful but legal)

We may project some attributes away instead of the standard way

This operator can be used to 1) rename attributes, and 2) permute attributes

Example of Projections

Airports

airport_name	country_name
Naples	Italy
London/Gk	England
London/Hw	England
Pisa	Italy
Venice/MP	Italy
Manchester	England
Kai Tak	Hong Kong
Changi	Singapore

Airports {country_name, airport_name}

country_name	airport_name
England	London/Gk
... ..	

Airports {country_name} a unary relation

country_name
Italy
England
Hong Kong
Singapore

(We could rename the attribute at the same time using the below expression:

```
Airport [RENAME country-name  
          AS country]  
)
```

Example of project

S { CITY }	<table><tr><th>CITY</th></tr><tr><td>London</td></tr><tr><td>Paris</td></tr><tr><td>Athens</td></tr></table>	CITY	London	Paris	Athens						
CITY											
London											
Paris											
Athens											
P { COLOR, CITY }	<table><tr><th>COLOR</th><th>CITY</th></tr><tr><td>Red</td><td>London</td></tr><tr><td>Green</td><td>Paris</td></tr><tr><td>Blue</td><td>Rome</td></tr><tr><td>Blue</td><td>Paris</td></tr></table>	COLOR	CITY	Red	London	Green	Paris	Blue	Rome	Blue	Paris
COLOR	CITY										
Red	London										
Green	Paris										
Blue	Rome										
Blue	Paris										
(S WHERE CITY = 'Paris') { S# }	<table><tr><th>S#</th></tr><tr><td>S2</td></tr><tr><td>S3</td></tr></table>	S#	S2	S3							
S#											
S2											
S3											

Fig. 6.5 Projection examples

3. Semantics - Natural Join

a1	b1	join	b1	c1	=	a1	b1	c1
a2	b1		b1	c2		a1	b1	c2
						a2	b1	c1
						a2	b1	c2

Let relations A and B have headings $\{X, Y\}$ and $\{Y, Z\}$ (where X, Y and Z are composite attributes).

The (natural) join of A and B : $A \bowtie B$

- is a relation with the heading $\{X, Y, Z\}$ and body consisting of the set of all tuples $\{X:x, Y:y, Z:z\}$ such that a tuple appears in A with X -value x and Y -value y and a tuple appears in B with Y -value y and Z -value z .

More about Natural Join

$A \text{ JOIN } B \text{ JOIN } C = (A \text{ JOIN } B) \text{ JOIN } C$
 $= A \text{ JOIN } (B \text{ JOIN } C)$ (associative)

$A \text{ JOIN } B = B \text{ JOIN } A$ (commutative)

If A and B have no attribute names in common, A
 $\text{JOIN } B = ?$
 $= A \text{ TIMES } B$

Example of Natural Join

Airports

airport_name	country_name
Naples	Italy
London/Gk	England
London/Hw	England
Pisa	Italy
Venice/MP	Italy
Manchester	England
Kai Tak	Hong Kong
Changi	Singapore

Stop

flight_number	airport_name	stop_number
BA383	Kai Tak	1
BA01	Changi	2

Stops JOIN Airports

flight_number	airport_name	stop_number	country_name
BA383	Kai Tak	1	Hong Kong
BA019	Changi	2	Singapore

Example of Natural Join

S

S#	SNAME	STATUS	CITY
S1	Smith	20	London
S2	Jones	10	Paris
S3	Blake	30	Paris
S4	Clark	20	London
S5	Adams	30	Athens

P

P#	PNAME	COLOR	WEIGHT	CITY
P1	Nut	Red	12.0	London
P2	Bolt	Green	17.0	Paris
P3	Screw	Blue	17.0	Rome
P4	Screw	Red	14.0	London
P5	Cam	Blue	12.0	Paris
P6	Cog	Red	19.0	London

S#	SNAME	STATUS	CITY	P#	PNAME	COLOR	WEIGHT
S1	Smith	20	London	P1	Nut	Red	12.0
S1	Smith	20	London	P4	Screw	Red	14.0
S1	Smith	20	London	P6	Cog	Red	19.0
S2	Jones	10	Paris	P2	Bolt	Green	17.0
S2	Jones	10	Paris	P5	Cam	Blue	12.0
S3	Blake	30	Paris	P2	Bolt	Green	17.0
S3	Blake	30	Paris	P5	Cam	Blue	12.0
S4	Clark	20	London	P1	Nut	Red	12.0
S4	Clark	20	London	P4	Screw	Red	14.0
S4	Clark	20	London	P6	Cog	Red	19.0

Fig. 6.6 The natural join S JOIN P

3. Semantics – θ -Join(θ 连接)

A TIMES B WHERE R θ S

e.g.

**((S RENAME CITY AS SCITY) TIMES
(P RENAME CITY AS PCITY))
WHERE SCITY>PCITY**

S#	SNAME	STATUS	SCITY	P#	PNAME	COLOR	WEIGHT	PCITY
S2	Jones	10	Paris	P1	Nut	Red	12.0	London
S2	Jones	10	Paris	P4	Screw	Red	14.0	London
S2	Jones	10	Paris	P6	Cog	Red	19.0	London
S3	Blake	30	Paris	P1	Nut	Red	12.0	London
S3	Blake	30	Paris	P4	Screw	Red	14.0	London
S3	Blake	30	Paris	P6	Cog	Red	19.0	London

Fig. 6.7 Greater-than join of suppliers and parts on cities

3. Semantics –equijoin(等值连接)

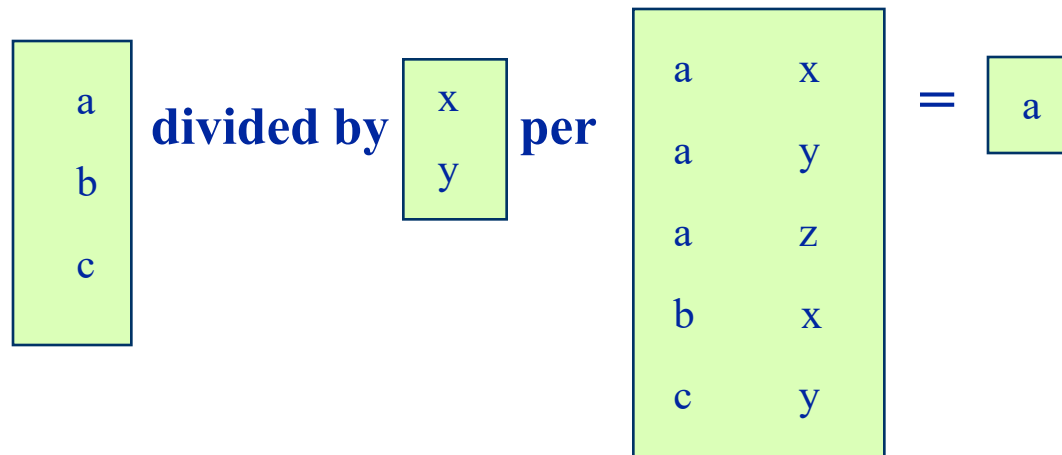
- If θ is "=", the θ -join is called an equijoin.

`S JOIN P`

—is equivalent to the following more complex expression:

```
( ( S TIMES ( P RENAME CITY AS PCITY ) )  
      WHERE CITY = PCITY )  
      { ALL BUT PCITY }
```

3. Semantics –Divide



Let relations A , B and C have headings $\{X\}$, $\{Y\}$, and $\{X, Y\}$ (where X and Y are composite attributes).

The division of A by B per C :

$A \text{ DIVIDEBY } B \text{ PER } C$

- is a relation with the heading $\{X\}$ and
- body consisting of all tuples $\{X:x\}$ such that a tuple $\{X:x, Y:y\}$ appears in C for *all* tuples $\{Y:y\}$ appearing in B .

Example of Division

Suppose that we wish to know airports to which we can fly from both London/Gk and London/Hw

Airports

destination
London/Gk
Pisa
Manchester
Kai Tak
London/Hw

DIVIDE BY

Airport_London

origin
London/Gk
London/Hw

PER

destination	origin
London/Gk	Naples
Pisa	London/Gk
Pisa	London/Hw
London/Gk	Pisa
Manchester	Venice/MP
Kai Tak	Manchester
Manchester	Kai Tak
London/Hw	Changi

=

Destin

destination
Pisa

‘Both’, ‘All’ etc. hint the division operator:
ex. *Get suppliers who supply all parts*

Example of Division

A DIVIDE BY B PER C

DEND		MED			
	S#	S#	P#	..	..
	S1	S1	P1	S2	P1
	S2	S1	P2	S2	P2
	S3	S1	P3	S3	P2
	S4	S1	P4	S4	P2
	S5	S1	P5	S4	P4
		S1	P6	S4	P5
		..	..		
DOR		DOR		DOR	
	P#		P#		P#
	P1		P2		P1
			P4		P2
					P3
					P4
					P5
					P6
DEND DIVIDE BY DOR PER MED					
	S#		S#		S#
	S1		S1		S1
	S2		S4		

Fig. 6.8 Division examples

3. Semantics – Associativity and Commutativity 结合律和交换律

**A UNION B UNION C = (A UNION B) UNION C
= A UNION (B UNION C) (associative)**

(Applicable to INTERSECT and TIMES but not to MINUS)

A UNION B = B UNION A (commutative)

(Applicable to INTERSECT and TIMES but not to MINUS)

Note that the Cartesian product operation of set theory is neither associative nor commutative, but the relational version as we have defined it IS both.

Note:

- Some of Codd eight operators are not primitive
- {restrict, project, product, union, difference} -
→ minimal set

4. Examples

S	S#	SNAME	STATUS	CITY	SP	S#	P#	QTY
	S1	Smith	20	London		S1	P1	300
	S2	Jones	10	Paris		S1	P2	200
	S3	Blake	30	Paris		S1	P3	400
	S4	Clark	20	London		S1	P4	200
	S5	Adams	30	Athens		S1	P5	100
P	P#	PNAME	COLOR	WEIGHT	CITY	S1	P6	100
	P1	Nut	Red	12.0	London	S2	P1	300
	P2	Bolt	Green	17.0	Paris	S2	P2	400
	P3	Screw	Blue	17.0	Rome	S3	P2	200
	P4	Screw	Red	14.0	London	S4	P2	200
	P5	Cam	Blue	12.0	Paris	S4	P4	300
	P6	Cog	Red	19.0	London	S4	P5	400

Fig. 3.8. The Suppliers and parts database(sample values)

4. Examples

1. Get supplier names for suppliers who supply part P2.

```
( ( SP JOIN S ) WHERE P# = P# ( 'P2' ) ) { SNAME }
```

2. Get supplier names for suppliers who supply at least one red part.

```
( ( ( P WHERE COLOR = COLOR ( 'Red' ) )  
      JOIN SP ) { S# } JOIN S ) { SNAME }
```

```
( ( ( P WHERE COLOR = COLOR ( 'Red' ) ) { P# }  
      JOIN SP ) JOIN S ) { SNAME }
```

4. Examples...

3. Get supplier names for suppliers who supply all parts.

```
( ( S { S# } DIVIDEBY P { P# } PER SP { S#, P# } )  
      JOIN S ) { SNAME }
```

4. Get supplier numbers for suppliers who supply at least all those parts supplied by supplier S2.

```
S { S# } DIVIDEBY ( SP WHERE S# = S# ( 'S2' ) ) { P# }  
      PER SP { S#, P# }
```

5. Get all pairs of supplier numbers such that the two suppliers concerned are "colocated" (i.e., located in the same city).

```
( ( ( S RENAME S# AS SA ) { SA, CITY } JOIN  
      ( S RENAME S# AS SB ) { SB, CITY } )  
      WHERE SA < SB ) { SA, SB }
```

4. Examples...

6. Get supplier names for suppliers who do not supply part P2.

```
( ( S { S# } MINUS ( SP WHERE P# = P# ( 'P2' ) ) { S# } )  
JOIN S ) { SNAME }
```

5. What is the Algebra for?

The fundamental intent of the algebra is to allow the writing of the relational expressions.

- Defining a scope for retrieval
- Defining a scope for update
- Defining integrity constraint
- Defining derived relvars
- Defining stability requirements
- Defining security constraints

5. What is the Algebra for?...

serve as a convenient basis for optimization

the expressions serving as a high-level, symbolic representation of the user's intent, can be transformed into the logically equivalent.

$((SP \text{ JOIN } S) \text{ WHERE } P\# = P\#('P2'))\{SNAME\}$

=

$((SP \text{ WHERE } P\# = P\#('P2')) \text{ JOIN } S)\{SNAME\}$

yardstick to measure the expressive power of some given language.

relational complete

关系运算有助于查询优化

- 通过对关系代数表达式的等价交换，减少多次扫描和读取文件以节省运行时间
- 尽可能早的执行选择运算，以期得到较少的中间结果
- 用连接运算代替乘积再选择运算
- 一连串的选择和一连串的投影同时运算，分开会多次扫描文件
- 预先计算并保存多次出现的子表达式结果
- 建立索引，提高关系连接的速度

课堂练习

Student

学号	姓名	性别	年龄	系
00301	李勇	男	20	CS
00302	刘晨	女	19	IS
00303	王明	女	18	MA
00304	张力	男	19	IS

Course

课程号	课程名	先行课	学分
1	数据库	5	4
2	数学		2
3	信息系统	1	4
4	操作系统	6	3
5	数据结构	7	4
6	数据处理		2
7	C语言	6	1

SC

学号	课程号	成绩
00301	1	92
00301	2	85
00301	3	88
00302	2	90
00302	3	80

课堂练习

求解：

1. 所有学生的姓名、性别；
2. 信息系学生的姓名、性别；
3. 选修了“数学”课的学生姓名、性别；
4. 同时选修了“数学”和“信息系统”课的学生姓名；
5. 至少选修了课程1、3的学生学号；
6. 上例中的学生学号、姓名、系；
7. 选修了课程2的学生学号；
8. 至少选修了一门其直接先行课为6的课程的学生姓名；
9. 选修了全部先行课为6的课程的学生姓名；
10. 选修了“刘晨”所选全部课程的学生姓名；
11. 年龄最大(or 成绩最高)的学生姓名；
12. 与“刘晨”同系的学生姓名；
13. 同系学生对。

关系运算（拓展）

- 关系模型与关系运算密切相关，关系运算理论是关系数据库技术的基础。
- 关系运算包括关系代数和关系演算。关系代数是关系为运算对象的一组高级运算的集合；关系演算基于数理逻辑谓词演算。
- 关系操作包括查询、插入、删除和修改，它们的数学基础是关系代数，关系代数又以集合理论为基础。
- 关系代数分为：五种基本运算并、差、乘积、选择、投影构成完备运算集，其他非基本运算可用五种基本运算合成。

1、关系代数

关系代数：

设关系R和S的目（属性或分量数目）分别为r和s，t为元组变量。

1) 并union运算：关系R和S相容（具有相同的模式，目及值域相同），两者并是由属于R和S的元组构成的集合：

$$R \cup S = \{t | t \in R \vee t \in S\}$$

注意：两个相容关系的列数相同，对应列同质；合并结果中若含有重复的元组，必须除掉多余的元组。

4
6

2) 差difference运算：关系R和S相容，两者差是由属于R但不属于S的元组构成的集合：

$$R-S = \{t | t \in R \wedge t \notin S\} \quad /$$

3) 乘积product运算：R为r元（目）关系，S为s元关系，两者乘积是一个(r+s)元的元组集合：

$$R \times S = \{t | t = \langle t_r, t_s \rangle \wedge t_r \in R \wedge t_s \in S\}$$

积的每个元组 $\langle t_r, t_s \rangle$ 均由原来的两个元组组成，前r个分量来自R，后s个分量来自S。

积的算法：用R的第i个元组分别与S的全部元组连接而成积的元组，因此乘积的基即元组个数等于R的基和S的基的乘积。

R和S有相同属性时，乘积的属性名不能相同，则用“R.属性名”和“S.属性名”以示区别。

4) **选择运算**：对关系进行水平分割，即从原关系中选取符合给定条件的那些元组：

$\sigma_F(R) = \{t | t \in R \wedge F(t) = \text{true}\}$, F为逻辑表达式。

Select ALL from R where F

F包含两种成分：运算对象如常数、元组分量；运算符如算术比较运算符、逻辑运算符。

5) 投影运算：对关系进行垂直分割，即从原关系中取出某些列并重新排序：

$\pi_A(R) = \{t[A] \mid t \in R\}$, A为R中的属性列。

Select A from R

投影结果往往含有重复的元组，必须除掉多余的。

6) 交运算：关系R和S相容，两者交是由属于R且属于S的元组构成的集合：

$R \cap S = \{t \mid t \in R \wedge t \in S\}$ 。

7) 连接：从两个关系模式中选取属性间满足一定条件的元组。

$$R \bowtie S = \{t \mid t = \langle t_r, t_s \rangle \wedge t_r \in R \wedge t_s \in S \wedge t_r[A] \theta t_s[B]\}$$

A和B分别为R和S上可比较的分量或属性（组）， θ 为比较运算符。

用R的第i个元组的A分量依次与S的每个元组的B分量进行 θ 比较，若满足则把S的那个元组接在R的第i个元组之后，作为连接关系的一个元组。

连接运算是在乘积运算的基础上增加了比较条件。

乘积运算在时间空间上开销大，优化连接可减少。

等值连接是 θ 为“=”的连接，它是从R和S的乘积中选取A和B属性值相等的那些元组。

$$R \bowtie S = \{t \mid t = \langle t_r, t_s \rangle \wedge t_r \in R \wedge t_s \in S \wedge t_r[A] = t_s[B]\}$$

自然连接运算：两个关系具有一个以上的公共属性（组）A，公共属性相等时连接对应元组构成一个新元组，从连接结果中除掉重复的属性列。

$$R \bowtie S = \{t \mid t = \langle t_r, t_s \rangle \wedge t_r \in R \wedge t_s \in S \wedge t_r[A] = t_s[A]\}$$

自然连接是一种特殊的等值连接。

8) 除法运算：关系 $R(X, Y)$ ，关系 $S(Y, Z)$ ，其中 X, Y, Z 为属性组， R 的 Y 和 S 的 Y 属性名可以不同，但必须出自相同的域集，则 $R/S = \text{关系} P(X)$ 。

9) 外连接运算：连接运算中如果把舍弃的元组也保存在结果中，无值的属性上填空值（NULL）。左外连接和右外连接。

10) 外部并运算：关系 R 和 S 不相容，外部并是由 R 或 S 的所有属性组成，没有具体值的属性为NULL。

2、关系演算

- 关系演算是以数理逻辑的谓词演算为基础，分为元组关系演算和域关系演算。
- 命题逻辑：能够用于判断的陈述句被称为命题。命题逻辑包含原子命题、用联结词及括号构成的分子命题，命题公式和函数，以及等价、蕴含、推理理论。
- 对原子命题的进一步解析产生了谓词逻辑。
- 谓词演算：原子命题是由一个谓词及若干个体两部分组成的，称为谓词命题。引入量词（全称和存在）的命题公式称为谓词命题公式，包含等价、蕴含、推理理论。

1) 元组关系演算

- 形式化表达: $\{t \mid P(t)\}$, t 是元组变量, P 是条件表达式, 其结果是一个满足公式 P 的所有元组 t 的集合或关系。
- 演算公式同谓词逻辑。
- 谓词运算符: $\exists$ -存在, $\forall$ -所有, $\neg$ -非, $\wedge$ -与, $\vee$ -或, $\rightarrow$ -如果...那么...
- 元组关系演算语言ALPHA。

2) 域关系演算

- 类似于元组关系演算，但变量不是元组而是元组中各个分量的域变量。
- 形式化表达： $\{ \langle t_1, t_2, \dots, t_k \rangle \mid P(t_1, t_2, \dots, t_k) \}$ ， $t_1, t_2, \dots, t_k$ 是元组变量 t 的各个分量。
- 域关系演算语言QBE。
- 元组演算表达式与域演算表达式可相互转换。

SQL 语言建立在关系代数和关系演算的数学基础上